After writing the integration tests, open a Terminal window, navigate to the Dispatcher Service project's root folder, and run the tests:

```
$ ./gradlew test --tests FunctionsStreamIntegrationTests
```

The next section will go through some points to consider regarding resilient integrations with messaging systems.

10.4.4 Making messaging resilient to failures

Event-driven architectures solve some issues affecting synchronous request/response interactions. For example, if you remove the temporal coupling between applications, you won't need to adopt patterns like circuit breakers, since the communication will be asynchronous. If the consumer is momentarily unavailable while the producer sends a message, it doesn't matter. The consumer will receive the message once it's up and running again.

In software engineering, there are no silver bullets. Everything comes at a cost. On the one hand, applications that are decoupled can operate more independently. On the other hand, you introduced a new component in your system that needs to be deployed and maintained: the message broker.

Assuming that part is taken care of by the platform, there's still something for you to do as the application developer. When an event happens and your application wants to publish a message, something might go wrong. Retries and timeouts are still helpful, but this time we'll use them to make the interaction between application and broker more resilient. Spring Cloud Stream uses the retry pattern with an exponential backoff strategy by default, relying on the Spring Retry library for imperative consumers and the `retryWhen()` Reactor operator for reactive consumers (the one you